

$$\begin{aligned}\text{Tr}[\mathbf{Y}^\top \mathbf{f} \mathbf{d}^\top] &= \mathbf{d}^\top (\mathbf{Y}^\top \mathbf{f}) \\ \text{Tr}[(\mathbf{f} \mathbf{d}^\top)^\top \mathbf{f} \mathbf{d}^\top] &= \|\mathbf{f}\|_2^2 \|\mathbf{d}\|_2^2\end{aligned}$$

Taking the derivative with respect to β and setting it to zero:

$$\frac{\delta}{\delta \beta} \|\mathbf{R}\|_F^2 = -2\mathbf{d}^\top (\mathbf{Y}^\top \mathbf{f}) + 2\beta \|\mathbf{f}\|_2^2 \|\mathbf{d}\|_2^2 = 0$$

This yields the closed form solution:

$$\beta = \frac{\mathbf{d}^\top (\mathbf{Y}^\top \mathbf{f})}{\|\mathbf{f}\|_2^2 \|\mathbf{d}\|_2^2} = \frac{\langle \mathbf{Y} \mathbf{d}, \mathbf{f} \rangle}{\|\mathbf{f}\|_2^2 \|\mathbf{d}\|_2^2} \quad (29)$$

Without loss of generality, we can assume $\mathbf{d}$ has unit norm.¹³

To gain intuition for this formula, consider a simplified toy setting where $f_i \in \{0, 1\}$ (latent either fires or doesn't) and $(\mathbf{Y} \mathbf{d})_i \in \{0, \alpha\}$ (the target contains the concept with magnitude α or not at all). In this case, the closed form simplifies to:

$$\beta = \frac{\sum_i (\mathbf{Y} \mathbf{d})_i f_i}{\sum_i f_i^2} \quad (30)$$

$$= \alpha \frac{\#\{i : f_i \neq 0 \text{ and } (\mathbf{Y} \mathbf{d})_i \neq 0\}}{\#\{i : f_i \neq 0\}} \quad (31)$$

$$= \alpha \cdot P(\text{concept present in target} \mid \text{latent active}) \quad (32)$$

This toy example illustrates that β captures both the magnitude α at which the concept appears in the target activations and the conditional probability that the concept is actually present when the latent fires. For a truly fine-tuning-specific latent, we expect this conditional probability to be near 0 for the base model activations (yielding $\beta \approx 0$) and near 1 for the fine-tuned model activations (yielding $\beta \approx \alpha$). In contrast, a shared latent should exhibit similar β values across both model activations, reflecting consistent presence of the underlying concept.

E.2 Detailed setup for Latent Scaling

We specify the exact target vectors $\mathbf{y}$ used in Equation (28) for computing the different β values to compute our chat-specificity metrics. To measure how well latent j explains the reconstruction *error*, we exclude latent j from the reconstruction. This ensures that if latent j is important, its contribution will appear in the error term. For chat-only latents, we expect distinct behavior in each model: no contribution in the base model ($\beta_j^{\varepsilon, \text{base}} \approx 0$) but strong contribution in the chat model ($\beta_j^{\varepsilon, \text{chat}} \approx 1$), resulting in $\nu_j^\varepsilon \approx 0$. In contrast, *shared* latents should have similar contributions in both models, resulting in approximately equal values for $\beta_j^{\varepsilon, \text{base}}$ and $\beta_j^{\varepsilon, \text{chat}}$ and consequently $\nu_j^\varepsilon \approx 1$.

$$\beta_j^{\varepsilon, \text{base}} : \mathbf{y}_i = \mathbf{h}^{\text{base}}(x_i) - \sum_{k, k \neq j} f_k(x_i) \mathbf{d}_k^{\text{base}} + \mathbf{b}^{\text{dec, base}} \quad (33)$$

$$\beta_j^{\varepsilon, \text{chat}} : \mathbf{y}_i = \mathbf{h}^{\text{chat}}(x_i) - \sum_{k, k \neq j} f_k(x_i) \mathbf{d}_k^{\text{chat}} + \mathbf{b}^{\text{dec, chat}} \quad (34)$$

To measure how well a latent j explains the *reconstruction*, we simply use

$$\beta_j^{r, \text{base}} : \mathbf{y}_i = \tilde{\mathbf{h}}^{\text{base}}(x_i) \quad (35)$$

$$\beta_j^{r, \text{chat}} : \mathbf{y}_i = \tilde{\mathbf{h}}^{\text{chat}}(x_i) \quad (36)$$

In a similar manner, we expect the fraction ν_j^r to be low for chat-only latents and around 1 for *shared* latents. For all of our analyses, we filter out latents with negative β^{base} values (L1: 46 in reconstruction and 1 in error, None in BatchTopK). These latents typically have low maximum

¹³By defining $f' = \|\mathbf{d}\|_2 f$ and $\mathbf{d}' = \mathbf{d} / \|\mathbf{d}\|_2$, we obtain an equivalent formulation with unit decoder norm.

activations and show a small improvement in MSE. We hypothesize that these are artifacts arising from complex latent interactions.

E.3 Additional analysis for Latent Scaling

Figure 10a and Figure 10b analyze the relationship between our scaling metrics (ν^ε and ν^r) and the actual improvement in reconstruction quality in the L1 crosscoder. For each latent, we compute the MSE improvement as:

$$\text{MSEImprovement} = \frac{\text{MSE}_{\text{original}} - \text{MSE}_{\text{scaled}}}{\text{MSE}_{\text{original}}}$$

where $\text{MSE}_{\text{scaled}}$ is measured after applying our Latent Scaling technique. We then examine the ratio of MSE improvements between the base and chat models, analogous to our ν metrics. The strong correlation between the ν values and MSE improvement ratios validates that our scaling approach captures meaningful differences in how latents contribute to reconstruction in each model.

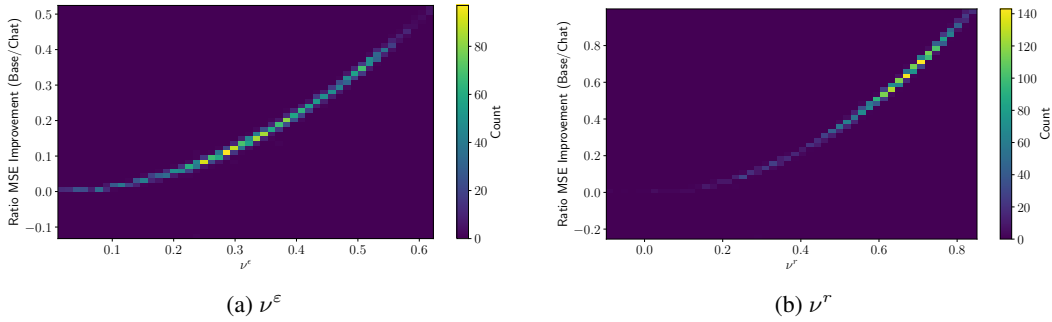


Figure 10: Comparison of the ratio of MSE improvement compared to the value of ν^ε and ν^r .

In Figure 11, we analyze the Latent Scaling technique by examining its relationship with the Δ_{norm} score. Specifically, we identify the 100 latents with the lowest ν^ε values and analyze their rankings according to the Δ_{norm} metric. As shown in Figure 11, there is limited correlation between the two measures - simply using a lower NormDiff threshold to identify *chat-only* latents produces substantially different results from our Latent Scaling approach.

F Cosine similarity of coupled latents.

As further evidence for Latent Decoupling occurring, we compute the cosine similarity between $\{\mathbf{d}_j^{\text{chat}}, j \in \text{chat-only}\}$ and $\{\mathbf{d}_j^{\text{base}}, j \in \text{base-only}\}$ revealing 109 (j, j_{twin}) pairs where $\text{cosim}(\mathbf{d}_j^{\text{chat}}, \mathbf{d}_{j_{\text{twin}}}^{\text{base}}) > 0.9$. To quantify activation pattern overlap between twins (j, j_{twin}) , we introduce an *activation divergence score* from 0 (always co-activate) to 1 (never co-activate) (see Section F.1). Figure 12 shows the divergence distribution across these pairs, highlighting that 60% of the pairs primarily activate on different contexts, with some pairs almost exclusively firing on different contexts (divergence of 1), while others exhibit substantial overlapping activations. This analysis demonstrates two important insights:

1. The Latent Decoupling phenomenon described in Section D, where the crosscoder learns a *base-only* and a *chat-only* latent that partially activate together instead of learning a *shared* latent, is empirically observed in practice.
2. Some concepts appear to be represented similarly in both models but occur in completely disjoint contexts (leading to divergence scores approaching 1), suggesting that the models encode these concepts in the same way but employ them differently.

Additionally, we find no pairs of *chat-only* latents and $\Delta_{\text{norm}} < 0.6$ latents with a cosine similarity greater than 0.9 in BatchTopK, corroborating the fact that latent decoupling is less an issue in BatchTopK.

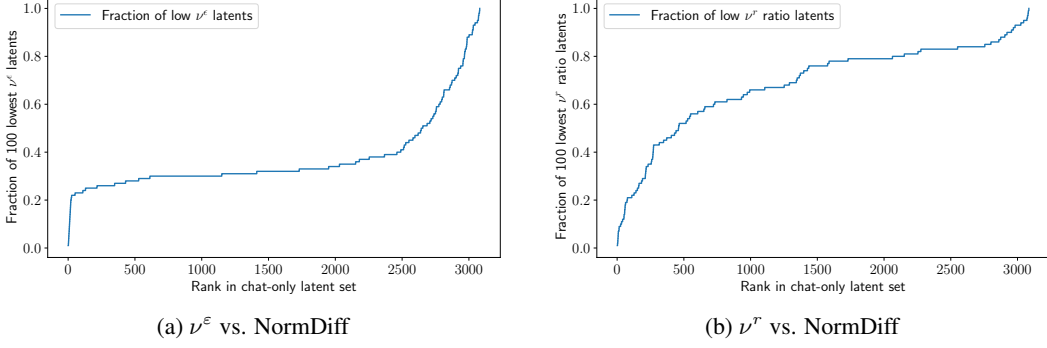


Figure 11: Comparison of latent rankings between ν and NormDiff scores. The lines shows the fraction of the 100 latents with the lowest ν values (x -axis) that have a rank lower than the given rank under the NormDiff score (y -axis).

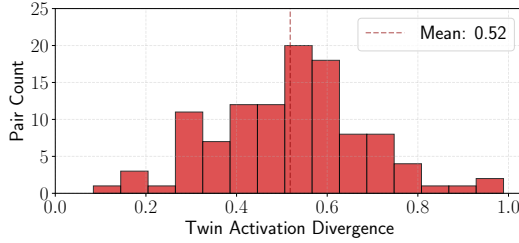


Figure 12: Distribution activation divergence over high cosine similarity (*chat-only*, *base-only*) latent pairs. 1 means that latents never have high activations ($> 0.7 \times \max_activation$) at the same time, 0 means that high activations correlate perfectly.

F.1 Detailed setup for activation divergence

In order to compute the activation divergence we compute for each pairs $p = (i, j)$, we first compute the max pair activation A_p on the training set D_{train} (containing data from LMSYS and FineWeb)

$$A_p = \max(A_i, A_j)$$

$$A_i = \max\{f_i(x)(\|\mathbf{d}_i^{\text{chat}}\| + \|\mathbf{d}_i^{\text{base}}\|), x \in D_{\text{train}}\}$$

Then the divergence Div_p is computed as follow

$$\text{Div}_p = \frac{\text{Single}_p}{\text{High}_p}$$

$$\text{Single}_p = \#\text{single}_i + \#\text{single}_j$$

$$\text{High}_p = \#(\text{high}_i \cup \text{high}_j)$$

where $\#\text{single}_i$ is the set of input $x \in D_{\text{val}}$ where i has a high activation but not j and high_i is the total number of high activations computed as follows:

$$\text{only}_i = \{x \in D_{\text{val}}, f_i(x) > 0.7A_p \wedge f_j(x) < 0.3A_p\}$$

$$\text{high}_i = \{x \in D_{\text{val}}, f_i(x) > 0.7A_p\}$$

G Causality experiments

G.1 Reproduction on LMSYS-CHAT

In Figure 13 we repeat the causality experiments from Section 3.2 for the L1 crosscoder on 700'000 tokens from the LMSYS-CHAT dataset, that the crosscoder was trained on. Note that while this dataset is much larger, the model responses are not generated by the Gemma 2 2b it model, and hence the model answers are out of distribution for this model. Since this dataset is much larger,

the confidence intervals are much smaller. The results are qualitatively similar to the ones on the generated dataset in the main paper.

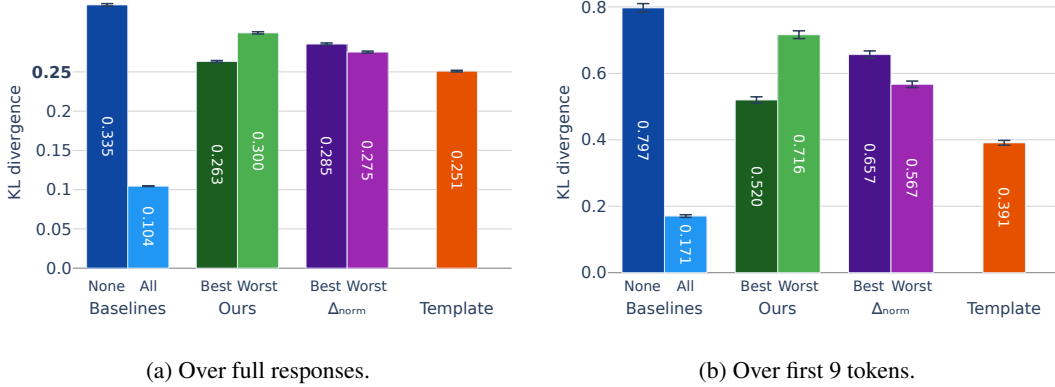


Figure 13: Comparison of KL divergence between different approximations of chat model activations on the LMSYS-CHAT dataset. We establish baselines by replacing either *None* or *All* of the latents. We then evaluate our Latent Scaling metric (*Ours*) against the relative norm difference (Δ_{norm}) by comparing the effects of replacing the top and bottom 50% of latents ranked by each metric (*Best* vs *Worst*). Additionally, we measure the impact of replacing activations only on template tokens (*Template*). We show the 95% confidence intervals for all measurements. Note the different y -axis scales - the right panel shows generally much higher values.

H Autointerpretability details

We automatically interpret the identified latents using the pipeline from Paulo et al. [2024]. To explain the latents, we provide ten activating examples from each activation tercile to Llama 3.3 70B [Grattafiori et al., 2024]. Latents are scored using a modified detection metric from Paulo et al. [2024]. We provide ten new activating examples from each tercile. Rather than comparing activation examples against randomly selected non-activating examples, we use semantically similar non-activating examples identified through Sentence BERT embedding similarity [Reimers and Gurevych, 2019] using the *all-MiniLM-L6-v2* model. To find these similar examples, we join all activating examples into a single string and embed it, then compute similarity scores against embeddings for each window of tokens to identify the most semantically related non-activating examples. This is a strictly harder task than scoring activation examples against a random set of non-activating examples.

I Reproducing results on other models

I.1 Llama models

We reproduce our experiments on both *Llama3.2 1B* and *Llama3.1 8B* models [Grattafiori et al., 2024]. Different from the Gemma models, the Llama models have a very different embedding for some of the template tokens. We replace several template tokens with single token alternatives:

- `<start_header_id>` is replaced with `\n\n`
- `<eot_id>` is replaced with `###`
- `<end_header_id>` is replaced with `###`

For Llama3.2 1B, we use the same training pipeline as the main paper with $\mu = 3.6e - 2$ for the L1 crosscoder, resulting in an L0 of 110 after training. We compare this to a BatchTopK crosscoder with $k = 100$. While this k value differs slightly, retraining would be computationally expensive, and the lower k actually disadvantages the BatchTopK crosscoder. The L1 crosscoder achieves 76.5% validation FVE while the BatchTopK crosscoder achieves 81.5%.

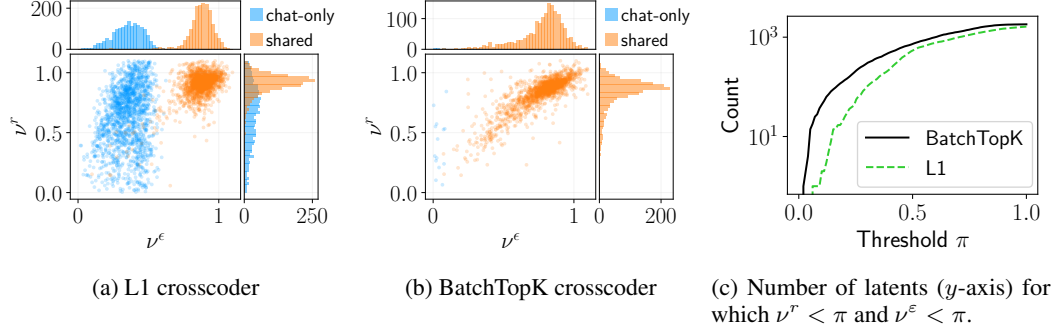


Figure 14: We compare how **Llama3.2 1B** *chat-only* latents are affected by the issues described in Section 2.2. Left/Middle: ν distributions for L1 and BatchTopK crosscoders, with each point representing a single latent. High ν^r values (y -axis) overlapping with *shared* distribution indicate Latent Decoupling (redundant encoding). High ν^ϵ values (x -axis) shows Complete Shrinkage (useful base latents forced to zero norm). Low values on both metrics identify truly chat-specific latents. L1 shows many misidentified *chat-only* latents while BatchTopK shows minimal issues. Right: Count of latents below a range of ν thresholds (x -axis), comparing 1844 L1 *chat-only* latents versus top-1844 BatchTopK latents sorted by Δ_{norm} .

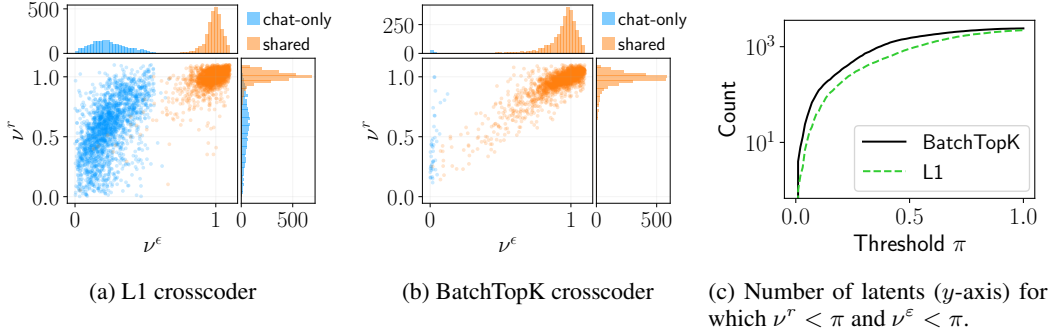


Figure 15: We compare how **Llama3.1 8B** *chat-only* latents are affected by the issues described in Section 2.2. Left/Middle: ν distributions for L1 and BatchTopK crosscoders, with each point representing a single latent. High ν^r values (y -axis) overlapping with *shared* distribution indicate Latent Decoupling (redundant encoding). High ν^ϵ values (x -axis) shows Complete Shrinkage (useful base latents forced to zero norm). Low values on both metrics identify truly chat-specific latents. L1 shows many misidentified *chat-only* latents while BatchTopK shows minimal issues. Right: Count of latents below a range of ν thresholds (x -axis), comparing 2442 L1 *chat-only* latents versus top-2442 BatchTopK latents sorted by Δ_{norm} .

For Llama3.1 8B, we use $\mu = 2.1e - 2$ for the L1 crosscoder, resulting in an L0 of 201, compared against a BatchTopK crosscoder with $k = 200$. For the BatchTopK crosscoder, we make two key modifications compared to the other models: 1) we initialize the encoder and decoder norms to 0.3 instead of 1.0 which is crucial for convergence, and 2) we anneal k from 1000 to 200 over 5000 steps to prevent dead latents. The L1 crosscoder achieves 76.6% validation FVE while the BatchTopK crosscoder achieves 81.5%. Due to computational constraints, we only use 10M tokens to train the latent scalars β .

Both models exhibit consistent patterns. The L1 crosscoders systematically overidentify *chat-only* latents:

- For Llama3.2 1B (Figure 14), the ν distributions reveal numerous misidentified *chat-only* latents in the L1 crosscoder, while the BatchTopK shows minimal issues. In Figure 14c we see that the BatchTopK crosscoder effectively identifies more truly chat-specific latents.
- The same patterns hold for Llama3.1 8B, as shown in Figure 15.